



Laboratório 07: Especialização de LLMs com LoRA e QLoRA

Objetivo do Laboratório: Construir um pipeline completo de *fine-tuning* de um modelo de linguagem fundacional (como o Llama 2 7B) utilizando técnicas de eficiência de parâmetros (PEFT/LoRA) e quantização (QLoRA) para viabilizar o treinamento em hardwares limitados.

Roteiro de Implementação

Passo 1: Engenharia de Dados Sintéticos (Dataset Generation)

- Vocês não vão baixar um dataset pronto. Utilizando a API da OpenAI (GPT-3.5 ou GPT-4), escrevam um script em Python para gerar um dataset sintético de instruções no domínio escolhido.
- O script deve gerar pelo menos 50 pares de prompt (pergunta/instrução) e response (resposta esperada).
- Dividam os dados (ex: 90% treino, 10% teste) e salvem o resultado no formato .jsonl.

Passo 2: Configuração da Quantização

- O treinamento tradicional (Full Fine-Tuning) exige a atualização de todos os parâmetros, o que estouraria a memória da GPU. Para resolver isso, utilizem a biblioteca bitsandbytes.
- Configurem o BitsAndBytesConfig para carregar o modelo base em **4-bits** utilizando o tipo de quantização nf4 (*NormalFloat 4-bit*) e o compute_dtype como float16.

Passo 3: Arquitetura do LoRA

- O LoRA congela a matriz original e treina apenas matrizes menores de decomposição. Utilizando a biblioteca peft, instanciem o LoraConfig com a tarefa CAUSAL_LM.
- **Hiperparâmetros obrigatórios:**
 - **Rank (r):** 64. (Dimensão das matrizes menores).
 - **Alpha (alpha):** 16. (Fator de escala dos novos pesos).
 - **Dropout:** 0.1. (Para evitar *overfitting*).

Passo 4: Pipeline de Treinamento e Otimização

- Utilizem o SFTTrainer da biblioteca trl para orquestrar o treinamento.





- No TrainingArguments, configurem a engenharia do otimizador para evitar picos de memória:
 - **Otimizador:** Usem o paged_adamw_32bit (AdamW paginado para transferir picos de memória da GPU para a CPU).
 - **Learning Rate Scheduler:** Usem o tipo cosine, para que a taxa de aprendizado caia suavemente seguindo uma curva de onda cosseno.
 - **Warmup Ratio:** Definam como 0.03 (os primeiros 3% do treino aumentarão a taxa gradativamente).
- Ao final do script, salvem o modelo adaptador (trainer.model.save_pretrained).

Critérios de Avaliação e Contrato Pedagógico

A avaliação deste laboratório seguirá estritamente as regras de integridade e submissão do ICEV:

1. Formato de Entrega e Versionamento:

- Todo o código-fonte (scripts Python ou Jupyter Notebook exportado) e o arquivo .jsonl do dataset devem ser enviados obrigatoriamente através de um repositório no **GitHub**.
- A versão final a ser avaliada deve ser marcada com a tag ou release "**v1.0**".

2. Funcionalidade e Estrutura do Código:

- O código deve rodar sem erros de compilação.
- A implementação deve conter explicitamente as configurações exigidas de LoRA (Rank, Alpha, Dropout) e o otimizador paged_adamw_32bit.

3. Política de Integridade e Uso de IA:

- **Uso Permitido:** Vocês podem usar IAs generativas (como ChatGPT ou GitHub Copilot) para *brainstorming*, pesquisa preliminar e geração de templates de código.
- **A Regra de Ouro:** Qualquer uso de IA deve ser revisado criticamente. É **obrigatório** inserir a seguinte nota no arquivo README.md do repositório: "*Partes geradas/complementadas com IA, revisadas por [Seu Nome]*".
- **Plágio (Nota Zero):** Submeter respostas ou códigos inteiros gerados por IA sem citação, ou "emprestar" o trabalho de um colega (mesmo alterando nomes de variáveis) configura quebra do contrato pedagógico. O trabalho receberá **nota 0** e o caso será registrado na coordenação (reincidência gera reprovação imediata).





4. Prazos e Penalidades de Atraso:

- A entrega deve ser feita no portal iCEV Digital até as **23h59** da data estipulada.
- 1 dia de atraso: penalidade de **-20%** na nota.
- 2 a 3 dias de atraso: penalidade de **-50%** na nota.
- Mais de 3 dias de atraso: **Nota 0** (salvo justificativa oficial via coordenação).

